

420-AS2-AS Final Project Report: Laptop Refresh & IT Asset Management SaaS Platform

1. Introduction (Anh)

High-level description

Our software company Ajax ACME, established in 2017, needs to upgrade our workforce's laptop fleet from the models that will be considered deprecated soon to a newer model that is current, that supports better tools and technologies. We are specialized in developing logistical solutions for small medium business to large enterprises to manage assets, both in terms of human (like a HRM platform) and physical assets.

Motivation

This project is to develop foundational internal processes and toolings that will be piloted, improved upon, and reused for future laptop upgrades.

Our motivations are:

- Be up to date with technologies and tools
- To continue receiving Windows updates and security patches
- To improve our internal user experience to boost productivity and to open new business opportunities

We are in a great position to develop a software solution that fits our needs as well as is able to solve the same problems that other companies face.

Background

Since our company was established in 2017, we have been using the Lenovo Thinkpad T470 with the Core i7 Intel CPU. Currently, we have in our production environment a total of 5000 of these laptops with 1000 more in backup supply for repairs and replacements, valued at US\$6M in total, that is US\$1,000 per unit. These were acquired as a one-time purchase contract with Lenovo.

The project is in response to Microsoft's deprecating Windows 10 and forcing upgrade to Windows 11. And the CPU requirements for Windows 11 are incompatible with our current laptop models in service. Therefore, the change will be making our laptops obsolete.

We need to take into considerations the following criteria:

- Budget, timeline, (human) resources
- What is the existing model - what is the next generation model
 - Need to consider different job functions require different models
- Automation of the project-related data so that we can reduce operational overhead
- Refresh period: laptop age before being upgraded
- Ensure employee's work is not disrupted
 - Set up the new laptops
 - Back up and transfer data
 - Send back old laptops
- Purchasing new laptops
- Recycling old laptops

2. Analysis of current business environment (Together)

- Positioning the product with respect to other products
Our product is tailored to our business environment, but it also has Software-as-a-Service extensible components that are positioned to sell to other companies.

In the current market, other software solutions on the market are IBM Maximo Asset Management, ServiceNow, SolarWinds, Azure Mobile Device Management, etc. We have not observed a product that is focused on the periodic refresh and upgrade of laptop models for businesses and streamlining the IT processes, in addition to IT asset management (ITAM).

With a market cap of US\$2.85B (according to Mordor Intelligence platform), the ITAM market presents high potential financial gains for our business. Just to remind, this is in addition to the potential costs saved in our future operations.

- Target public of the product
We are targeting the SMB and large enterprises that issue their workers laptops and have the similar needs to our internal processes. These potential customers are looking for an off-the-self solution so as to not reinvent the wheel and to save operational costs within their own organization.

These companies might be in the following, but not limited to, sectors:

- Banking and Finance: Desjardins, BMO, RBC, CIBC, Scotia Banque, AMEX, Wealthsimple, etc.
- IT and Software
- Heavy industries
- Defense, government and contractors

- The direct usage of the platform would be primarily targeted towards the following roles in an organization: IT operation and security (e.g. system administrators, IT support, information security/SOC analysts), auditors (e.g. Governance Risk Compliance - GRC) and business analysts (e.g. financial analysts, project managers).

3. Development Methodology (Together)

- Applicable methodologies (pros and cons)
Rationale: Since the software we are looking to develop, use internally, and sell externally, is a Software-as-a-Service model, the revenue source would be charging contracts with companies as well as taking into account the internal customer base usage and costs.

See the matrix below for pros and cons breakdown.

Development Methodology	Pros	Cons
Agile	Frequent feedback; early increments; adaptive; continuous testing	Uncertain scope; requires high involvement; estimation difficulty; potential churn
Waterfall	Clear requirements; predictable timeline; strong documentation	Inflexible; late testing; no early product; high risk if requirements are wrong
Incremental	Functional slices; easier debugging; flexible; faster initial output	Needs strong architecture; integration complexity; scope creep; NFRs may be delayed

- Decision and recommendation
Agile fits a SaaS platform by enabling rapid, low-risk iteration through short cycles, CI/CD, feature flags, and telemetry-driven validation. It delivers continuous value to both internal users and external customers, supports controlled rollout strategies, integrates security and compliance checks directly into pipelines, and maintains alignment through frequent feedback. This reduces lead time, lowers release risk, improves prioritization based on real usage data, and keeps the product responsive while retaining governance and predictable quality.

4. Description of Project (Together)

Detailed description of the solution

The solution is a software-as-a-service (SaaS) platform that will begin as a web-based full-stack application.

The application contains the following components that are characteristic of a SaaS platform.

Software Architecture

For a SaaS platform, the architecture follows the industry commonly-used pattern:

Service-oriented Architecture (SoA) which would be highly scalable and resilient, in contrast with the traditional monolithic architecture where the software layers are tightly coupled.

While there are separations between the different backend services, we decide to keep the frontend monolithic to avoid high upfront development effort. However, we still need to design and structure the frontend in a somewhat scalable way to future-proof ourselves from technical debt and impossible refactors.

A SoA platform will leverage the following design patterns from the design and development stages to the deployment stage.

Message-based data exchange

Services of the application will use messages to communicate with each other. Messages are produced by originating services, sent to a message queue, and consumed by the destination service. The primary advantage that we want to leverage is the resilience of this data exchange method. It makes sure that the services are atomic in processing data and the downstream services do not rely on the successful execution of the upstream services. In other words, failure in one service does not constitute a cascaded failure down the chain.

For example, a critical failure in adding a new asset does not halt or break the entire backend (i.e. a cascade effect) or a data I/O (retrieving and updating assets by different users) failure does not lock the database connection for everyone else.

Distributed system and services

The platform uses a service mesh to automatically detect, register, and bind service instances (also referred to as a node in a mesh). The service mesh, coupled with the message broker and queue, will ensure data integrity and delivery. The service mesh itself acts as a load balancer as well, so the workload for our service nodes are distributed evenly, using strategies such as (weighted) round robin, just-in-time resource-based, or geo-based (nearest node on the network), avoiding unpredictable load and degraded performance.

Software Modules

Frontend

- Common UI components:
 - The application header: displays the branding (maybe customized for enterprise clients), currently logged in user, and navigations.
 - The application footer: displays the app branding (same), normalized navigation links, copyright and legal information, app version and environment, other details if needed.
 - Mobile viewport versions of the header, footer, and navigations to ensure mobile responsiveness.
- The main page shows an overview of the IT inventory: assets deployed, compliance status, devices up for refresh, charts that give general statistics.
- Asset management dashboard: CRUD on the assets and assignment to users.
- Financial operations dashboard: perform business intelligence analysis on the assets to optimize costs.
- Upgrade/Refresh dashboard: Initiate and perform the process of upgrades/refreshes for the user base.

Backend

Before moving on to manifesting the backend components, we need to define what is a backend service and repository.

- **Service:** not to be confused with a service in the service mesh, contains the business logic and the data processing logic. The backend service in this case will also expose API endpoints that are used by the frontend to perform business logic, helping the user achieve their goal. Will sometimes be referred to as the API layer.
- **Repository:** is the direct communication layer with the database, responsible for connecting and executing database queries. The repository helps decouple the service from the database schema, making the service agnostic, and masking the specificity of interacting with a particular database (e.g. postgres, mysql, SQL server, Oracle). This abstraction reduces technical debt when we want to integrate with different databases.

Note that each backend “service” has a service and repository component.

Services

- Asset
- User
- Authentication and Authorization (AuthN/Z)
- Administration

Since our backend services are decoupled, self-contained, atomic and specialized, we can scale horizontally to handle peak hours or spikes in usage and sometimes scale-to-zero to optimize compute resource costs. These service instances/nodes may be provisioned as much as needed. We can also easily perform rolling upgrades or rollbacks in case of critical failures

without disrupting the entire system. With this, we can then easily define Service Level Agreements (SLA) and Service Level Objectives (SLO) for our stakeholders and clients.

Message Broker and Queue

The message broker and queue, also collectively called the data backplane, handle data exchanges in the form of messages between services in the application. When a service produces data (producer), the data is packed into a message and passed along the message bus to the message broker. The message broker will queue the message and the service which consumes the data (consumer) will process the message intended for it using the FIFO strategy.

Database

We are using a Structured Query Language (SQL) database for data persistence. A SQL database offers a robust data storage option as well as extensive features for complex data operations.

Our database is configured at the instance level. An instance will be provisioned for a client organization, be it our internal company or external clients. Each instance is sharded, partitioned, and replicated for performance and data security.

An instance database for each organization will allow us as the provider to customize the database to meet security compliances to the specific client.

We will also utilize a fast in-memory cache database for user session management, rate limiting and data exchange.

Application Usage Workflows

- User can navigate to the platform using their client of choice (web or desktop app)
- User can sign in to the application using their organization-defined method
- User must be authenticated and authorized to access the platform
- User should be redirected to the main page after successful AuthN/Z
- User may be shown the Overview dashboard with general metrics for the assets
- User may view a breakdown of each metric and charts
- User may navigate to the user and asset management dashboard, financial operations dashboard, upgrade/refresh dashboard.
- User may view the platform operational guides
- User may log out to end their usage session

Typical Usage Process

Asset management dashboard

- See what assets are in the organization and the usages
- Add or remove assets
- Assign a user to an asset
- Disable an asset
- Reset an asset

- Quarantine an asset in case of security incident

User management dashboard

- Add or remove user
- Sync users from a directory service (LDAP, Microsoft Entra, Google Workplace Directory, etc.)

Upgrade/Refresh dashboard

- Perform singular upgrade
- Perform batch upgrade
- Onboard assets
- Recycle assets
- View upgrade progress: Started, Preparing asset, Shipped, Delivered, Received old asset, Recycled old asset, Finish
- Print shipping label for asset to user

Scope (what is in and what is out)

Technology

Frontend

- Web: React/Nextjs or Vue/Nuxtjs
- Desktop: Electronjs (cross-platform), .NET Windows App (Windows), Swift App (Apple ecosystem), Flutter (Android), ApplImage (Linux, distro agnostic)

Backend

- Services: pick one between Nestjs (TypeScript, Expressjs or FastAPI-based), Golang (Gin), or Python (Django, Flask, FastAPI), Rust (Poem, Rocket)
- ORM: Drizzle ORM
- Data exchange protocol: HTTP or gRPC
- Data payload: Protobuf or JSON depending on the data exchange protocol
- Message Queue: Apache Kafka or RabbitMQ

Database

- Long-term data storage: PostgreSQL
- In-memory cache: memcache or Redis

Network and Security

- SSL/TLS terminated connection between the modules – data encryption in transit.
- Data encrypted at rest and decrypted just-in-time for processing.

Assumptions, Risks, Constraints and Contingence Plans

Assumptions: stable cloud infrastructure, teams adopt new processes

Risks: rollout disruption (e.g. theft, dead on arrival), integration challenges, security concerns

Constraints: budget, resources, compliance and regulations

Contingency: phased rollout, pilots, rollback strategies

Preliminary Timeframe

Reaching the MVP state in 5 months

Break-even point:

Best case: 8.5 months

Worst case: 16.5 months (1.25 years)

Preliminary Cost estimates

Immediately earmarked \$1M (1% of annual net revenue) for the MVP stage of the product lifecycle

Personnel cost: \$240,000 (base) + \$160,000 (OT, bonus) (24% + 16%)

Infrastructure cost: \$100,000 (10%)

Toolings & Licensing cost: \$100,000 (10%)

Operational & Management cost: \$300,000 (30%)

Budget Overhead: \$100,000 (10%)

Additionally commit \$1M more; needs evaluation, retrospection; needs C-Suite approval

Hard-capped at \$2.5M in total; considered failed and deprecated when MVP is not satisfied at this point

5. System Analysis and Design (Refer to attached Powerpoint)

- A. Use case diagrams (Kelly)
- B. Activity diagrams (Kelly)
- C. Use case textual descriptions (Almendra)
- D. Class diagrams (Almendra)
- E. Sequence diagrams (Anh)
- F. Preliminary Product Backlog (with priorities and complexities) (Anh)

6. Conclusion

Our project turns a painful laptop refresh into a reusable SaaS platform that helps IT teams plan, execute, and track upgrades with clear workflows. By using Scrum and short iterations, we can deliver small features quickly, test them with real users, and improve based on feedback. With phased rollout, pilots, and security controls, we reduce rollout risk and integration issues. In the end, this platform not only solves our own refresh problem but can also be offered to other companies facing the same challenge.